Security (System) Engineering with Generative AI

Homework Assignment no. 6

Student name: Brehon Parker

FAU Email: bparker2012@fau.edu

Algorithm:

I. Edit above your student name and homework number.
II. Edit below the missing information between < and >.
III. During development, for every significant task given to the agent you must:
   A. Take a screenshot of your VSCode window clearly showing the prompt you gave to the agent and the response it gave you back. Your FAU ID is the email address part excluding @fau.edu and it must be visible on the screenshot.
   B. Take a screenshot of your VSCode window clearly showing at least the top few lines of code that were generated or modified by the agent. Your FAU ID must be visible on the screenshot.
   C. Insert the screenshots into the homework Word document, along with the task prompt given to the agent pasted as regular text.
IV. When finished, convert the Word document to PDF and upload on the homework Canvas page.

Screencast Recording URL: *Pending student recording and unlisted YouTube upload*

Github Reposity URL:
https://github.com/bparker2012-cyber/gensec-code/tree/codex/hw6-secure-notes/hw6

Application name: *Secure Notes*

Application Description: *Secure Notes is a local Flask application generated with OpenAI Codex. Separate user accounts can create, read, edit, and delete persistent private notes stored in SQLite.*

Development task and coding agent

Coding agent: OpenAI Codex, Codex desktop app. I accepted the secure notes proposal and requested methodical GitHub commits. The following task text and replies come from the actual Codex conversation export. Prepared replay prompts are identified separately.

Accepted application proposal

My recommendation is a secure notes application built with Codex: login, private notes, SQLite storage, password hashing, CSRF protection, and input validation. It connects naturally to Lab 6 and gives us clear security demonstrations that fit the recording limit.

Actual task prompt

lets get it donev:) remmeber github commits should be methodiically as we would typically do....

Actual Codex response

I’ll build the secure notes app, save the prompts and evidence, and prepare the submission documents and recording script. I’ll commit in clear stages: scaffold, working app, security checks, then submission materials.

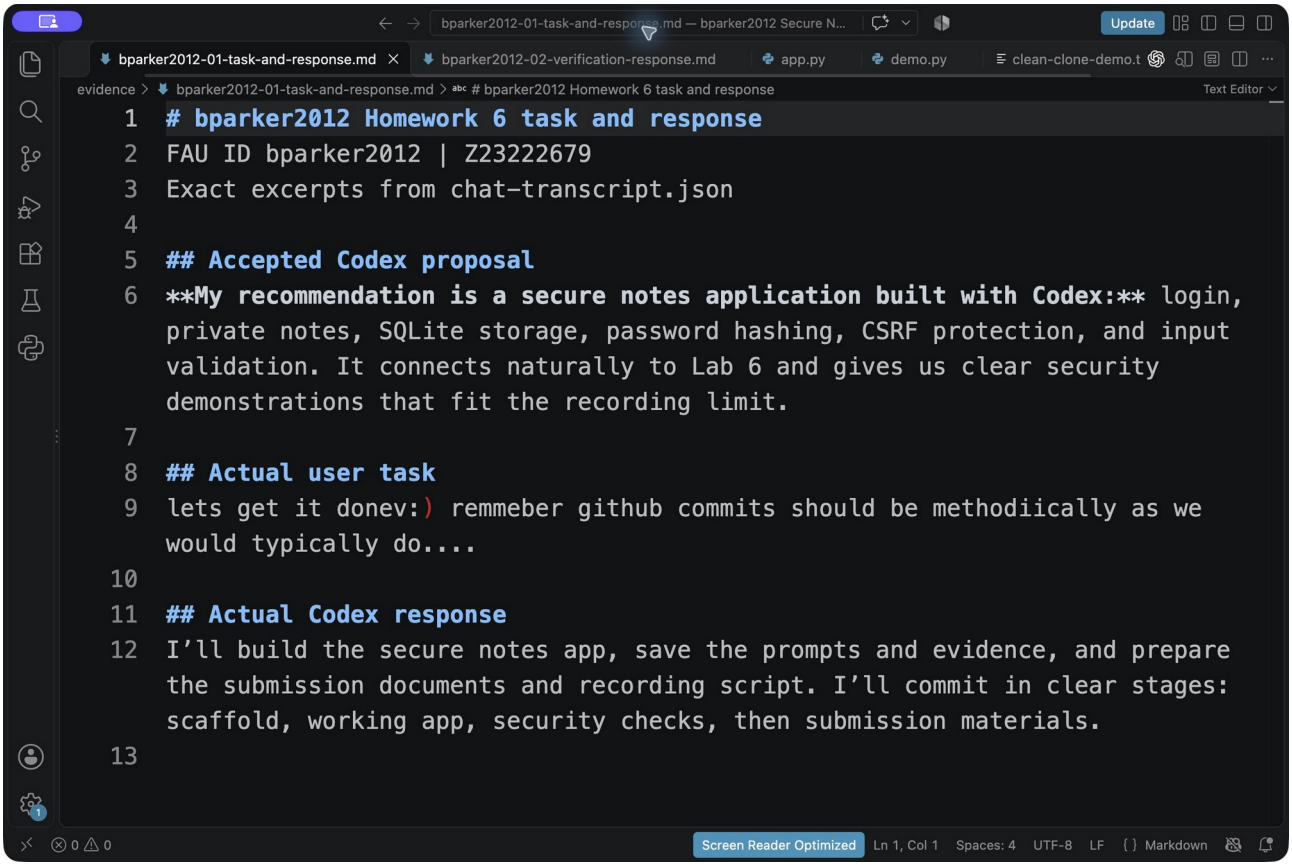


Figure 1. VS Code displays exact excerpts of the real task and Codex response. The file identifies its conversation-export source and the FAU email ID is visible.

Standalone prompts for recreation

These self-contained instructions were prepared from the agreed task for the required screencast replay. They are not additional historical user messages. The complete wording and provenance are preserved in prompts.txt.

Prompt 1 Build the application

Build a small Python 3.13 Flask application named Secure Notes inside hw6. Use app.py with a create_app factory, Jinja templates, local CSS, and SQLite. Implement registration, login, logout, and create/read/edit/delete private notes. Use a clean, readable interface with a sidebar, note list, editor, clear errors, and a footer showing Brehon Parker, bparker2012@fau.edu, and Z23222679. Keep the app usable without external APIs, JavaScript, or internet services. Treat every form field, URL, and cookie as adversarial. Use Werkzeug scrypt password hashing, parameterized SQL, owner-scoped queries on every note route, Jinja autoescaping, per-session CSRF tokens on every POST including auth and logout, session clearing at auth transitions, bounded input sizes, a persistent random signing key outside Git, HttpOnly/SameSite cookies, no-cache responses, security headers, trusted loopback hosts, and debug mode off. Add a bounded SQLite-backed login/registration rate limit. Store only an opaque session token in the signed cookie and its hash server-side so logout revokes captured cookies. Run on 127.0.0.1:5006 by default, with optional Secure cookies for HTTPS. Document that note content is plaintext at rest, local HTTP is a classroom mode, and this is not a production deployment. Pin dependencies with pyproject.toml and uv.lock, and ignore databases, signing keys, virtual environments, and caches. Keep the scope small enough to explain in a five-minute screencast.

Prompt 2 Verify security and prepare the demonstration

Exercise the real application and add meaningful automated tests for private note isolation, anonymous access, CSRF rejection, stored XSS escaping, SQL injection attempts, password hashing, revoked sessions, malformed and oversized input, security headers, and rate limiting. Fix any failures without weakening the controls. Add a reproducible --demo security-check command using temporary data and Flask's test client, clearly labeled as in-process checks. Do not add an HTTP test endpoint. Supply setup/run/test instructions, a security overview with honest limitations, and a short recording script that follows the required order: git clone, prompt walkthrough, live regeneration in a fresh directory, security explanation, functionality and limitations. Preserve real evidence; never fabricate a model response, screenshot, recording URL, or submission.

Generated application code

Codex generated app.py, local Jinja templates, CSS, and a locked dependency environment. Registration, note creation, reading, editing, deletion, and sign-out passed the initial smoke check before the application commit was pushed.

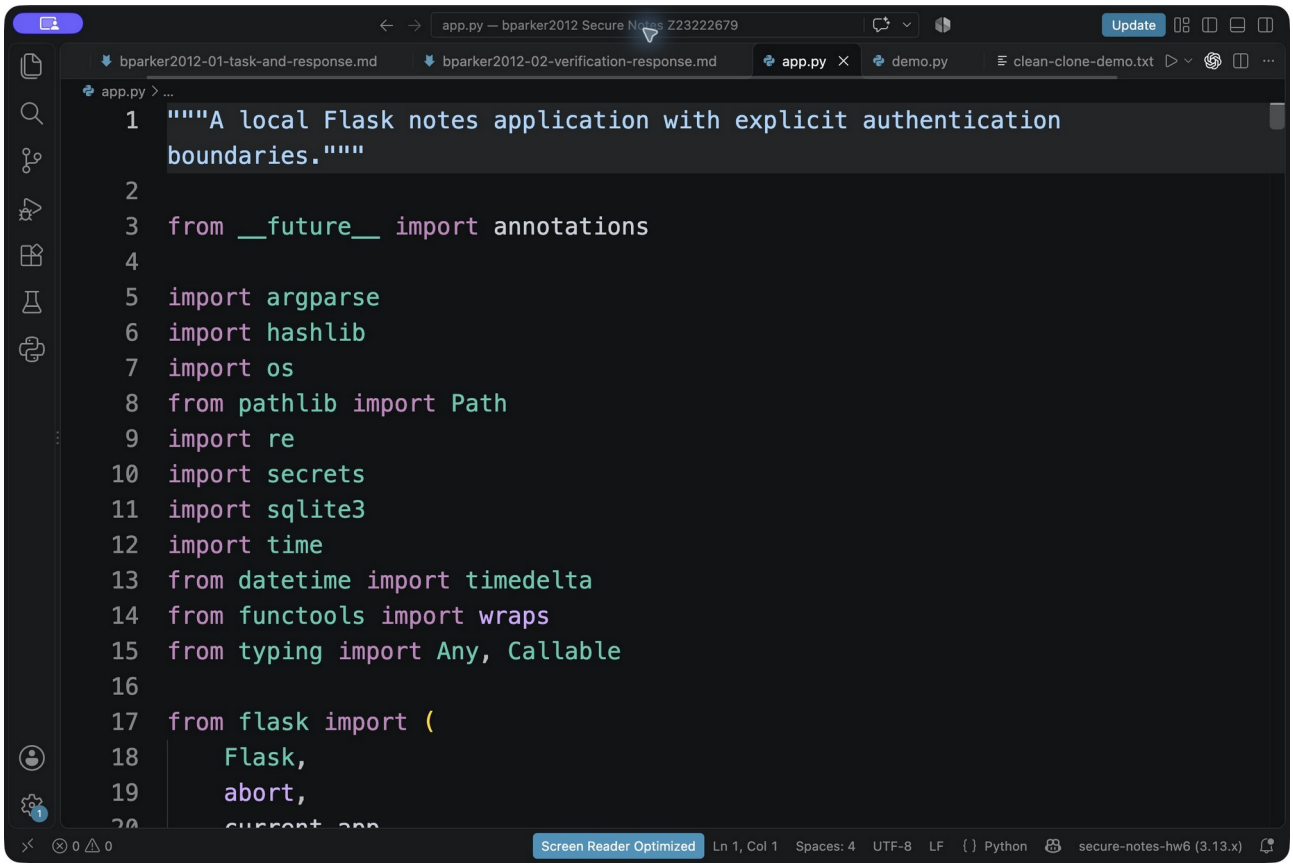


Figure 2. Top lines of the generated Python application in VS Code, with bparker2012 visible in the window title.

The source remains readable and divided into a small application factory, database helpers, authentication checks, and note routes. The UI requires no JavaScript or remote services.

Security verification and agent replies

All 59 security tests pass, including account isolation, CSRF rejection, HTML escaping, SQL injection attempts, and replaying a signed-out cookie. The checks also caught two error-handling bugs with hostile hostnames and oversized note IDs; both are fixed. That stage is committed and pushed.

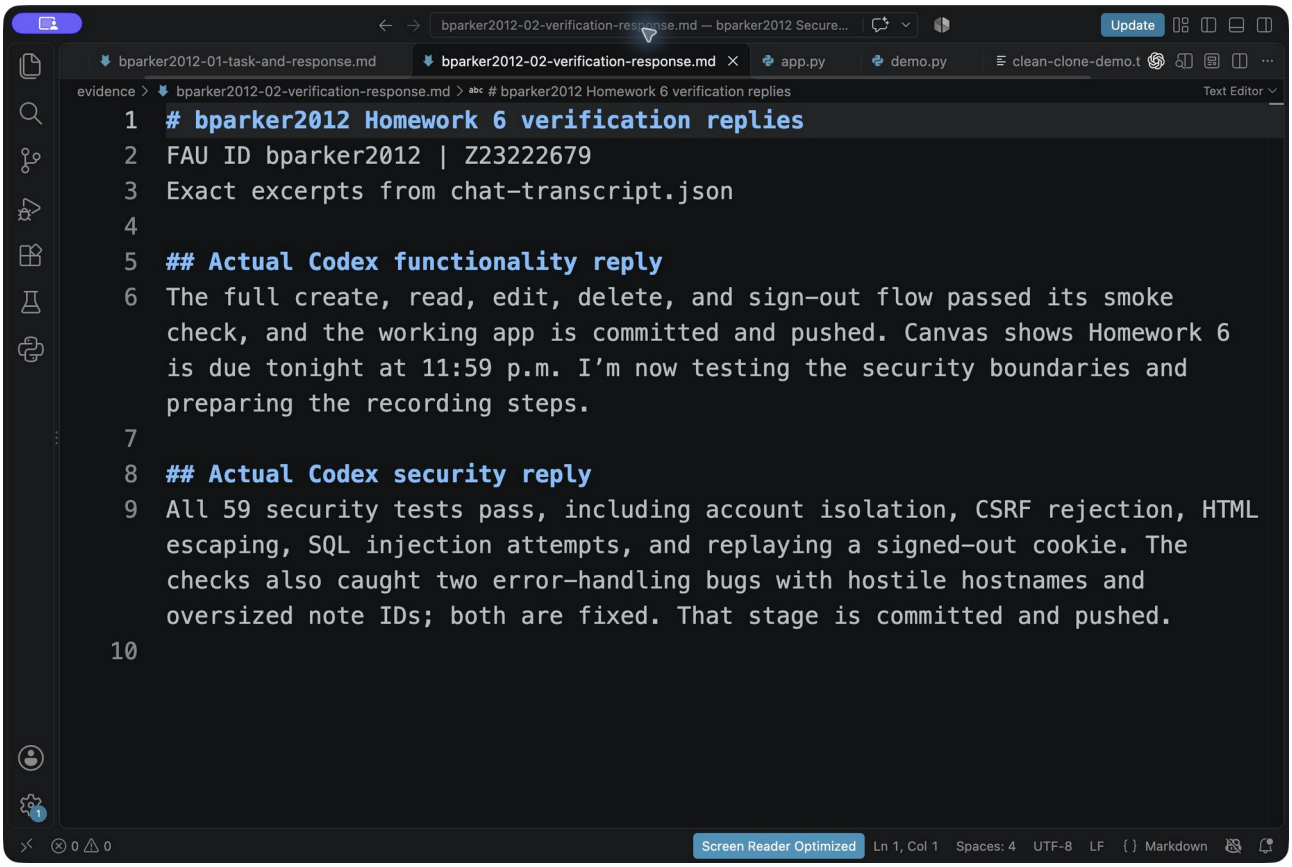


Figure 3. Exact Codex verification replies in a labeled VS Code transcript view.

Testing identified an error page that tried to build URLs after a hostile Host was rejected, and oversized URL IDs that could overflow SQLite. The error handler now uses the framework response for rejected hosts, and a bounded route converter rejects invalid IDs before conversion or database access.

Generated verification code

Codex added adversarial-input tests and a disposable demonstration. The --demo command calls actual Flask routes and SQLite through the test client. It uses temporary synthetic data and is separate from the live browser demonstration.

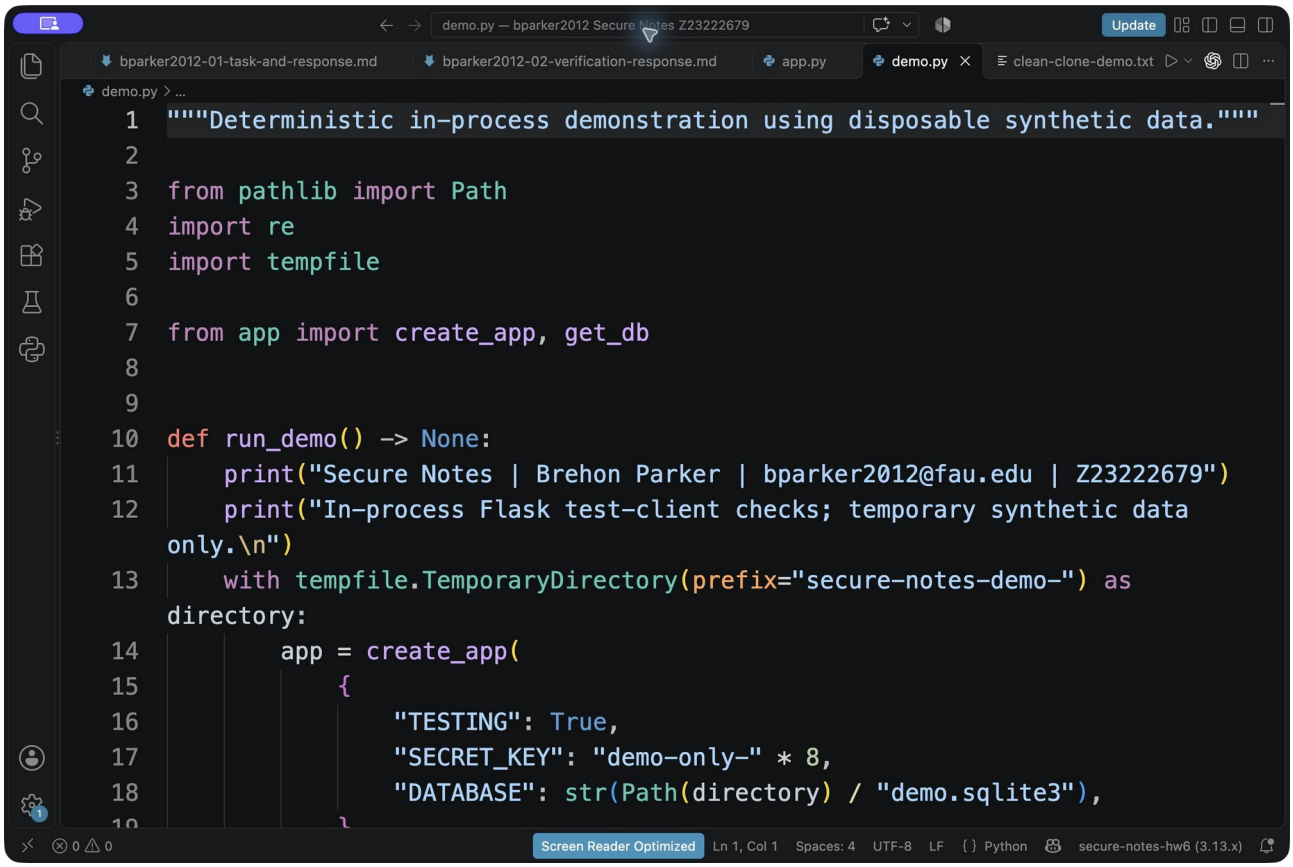


Figure 4. Top lines of the generated demonstration code, with the FAU ID visible.

Private note ownership

Every note lookup includes both the note ID and the authenticated user ID. Update and delete statements repeat the owner condition. A second account cannot list, open, edit, or delete the first account's note; these requests are covered by the passing test suite.

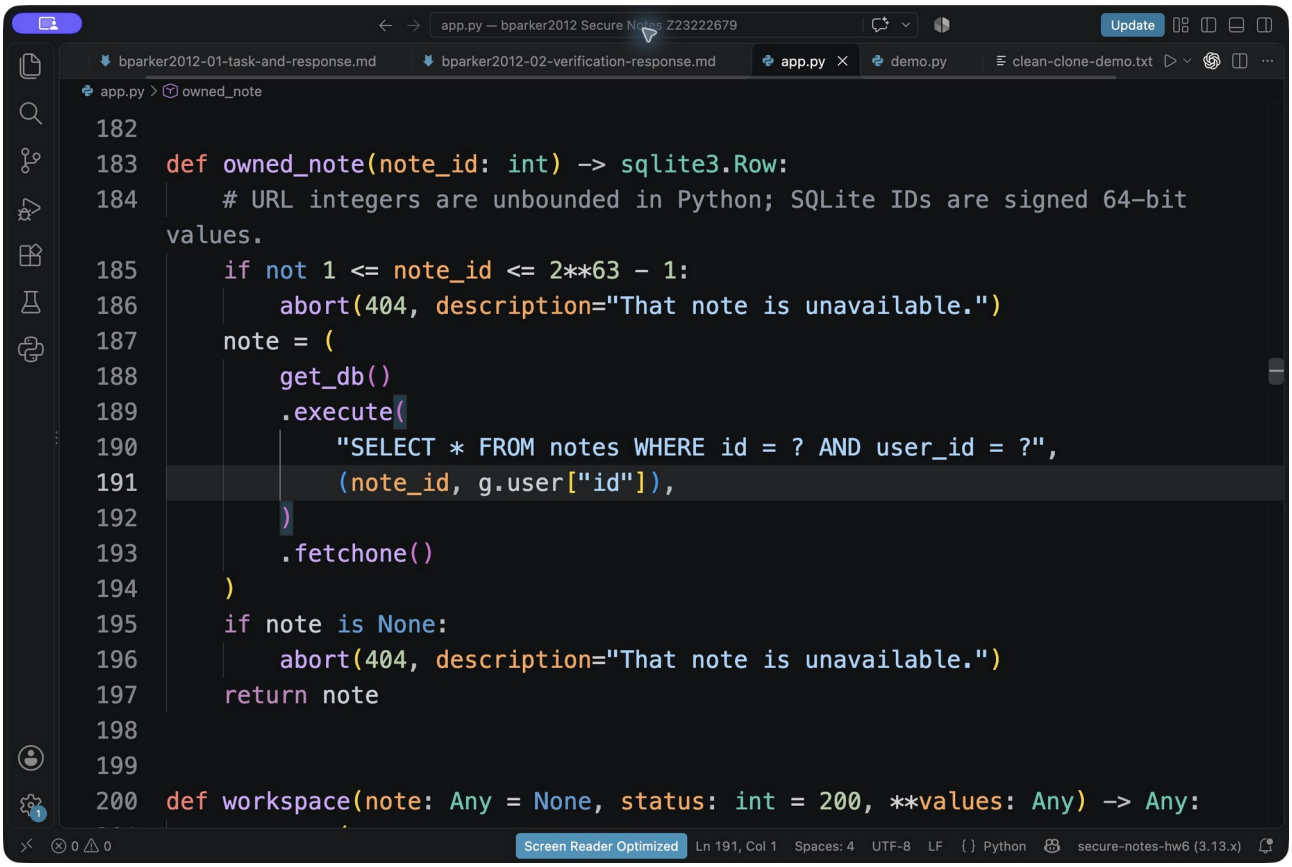


Figure 5. The generated owner-scoped query and bounded note ID validation in app.py.

CSRF and authentication sessions

Every POST, including login, registration, and logout, verifies the session CSRF token. Authentication transitions clear session state and rotate the token. An opaque token in the signed cookie maps to a hashed token in SQLite, allowing logout and subsequent login to revoke captured old cookies. Sessions expire after 30 minutes.

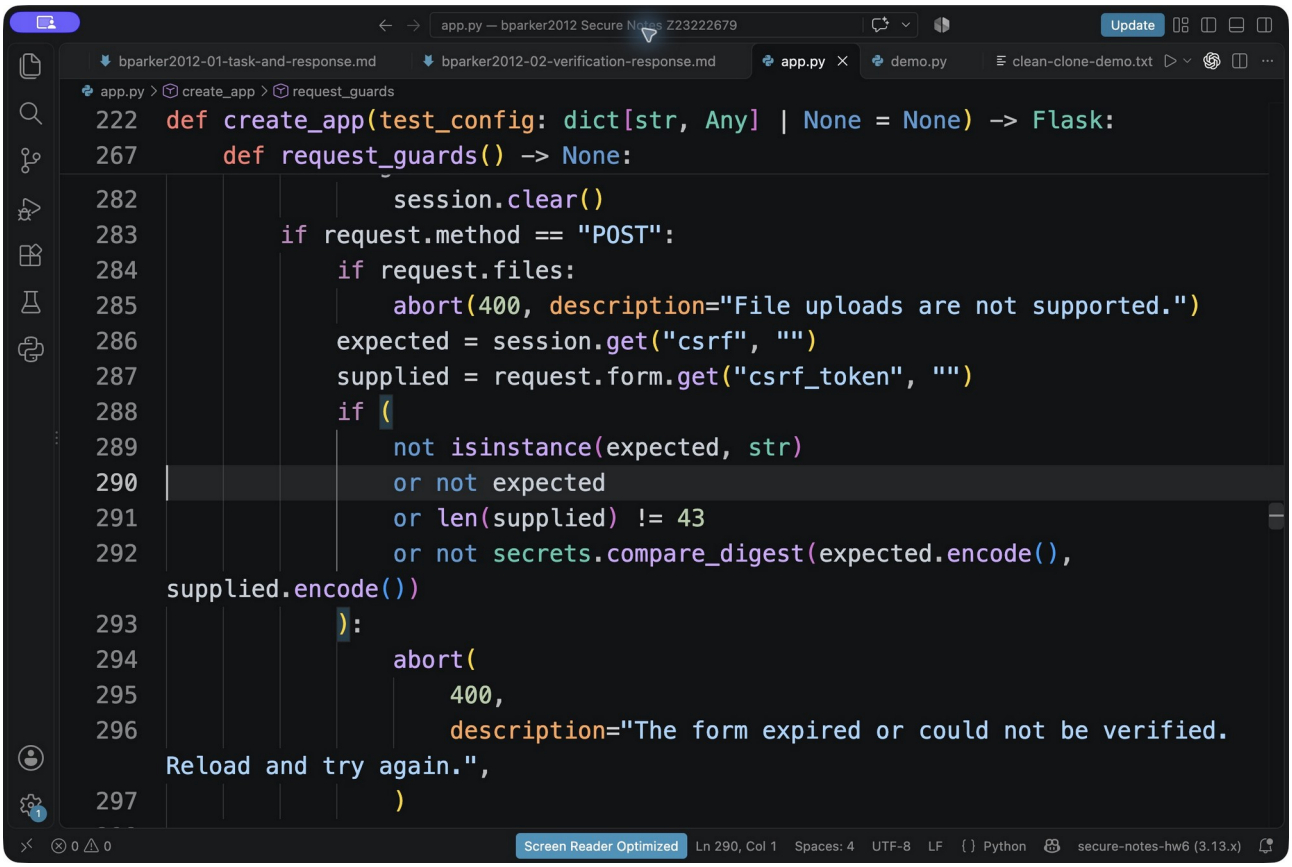


Figure 6. Request guards load the authenticated user and verify all POST form tokens.

Verification from a fresh GitHub clone

A fresh clone of remote commit 0f69a3998d4a installed the locked dependencies, passed all 59 tests, ran the security demo, and passed Ruff checks. This verifies the checked-in application from a clean checkout; it does not establish a new AI regeneration.

```
.....................................................................  [100%]
59 passed in 8.92s
```

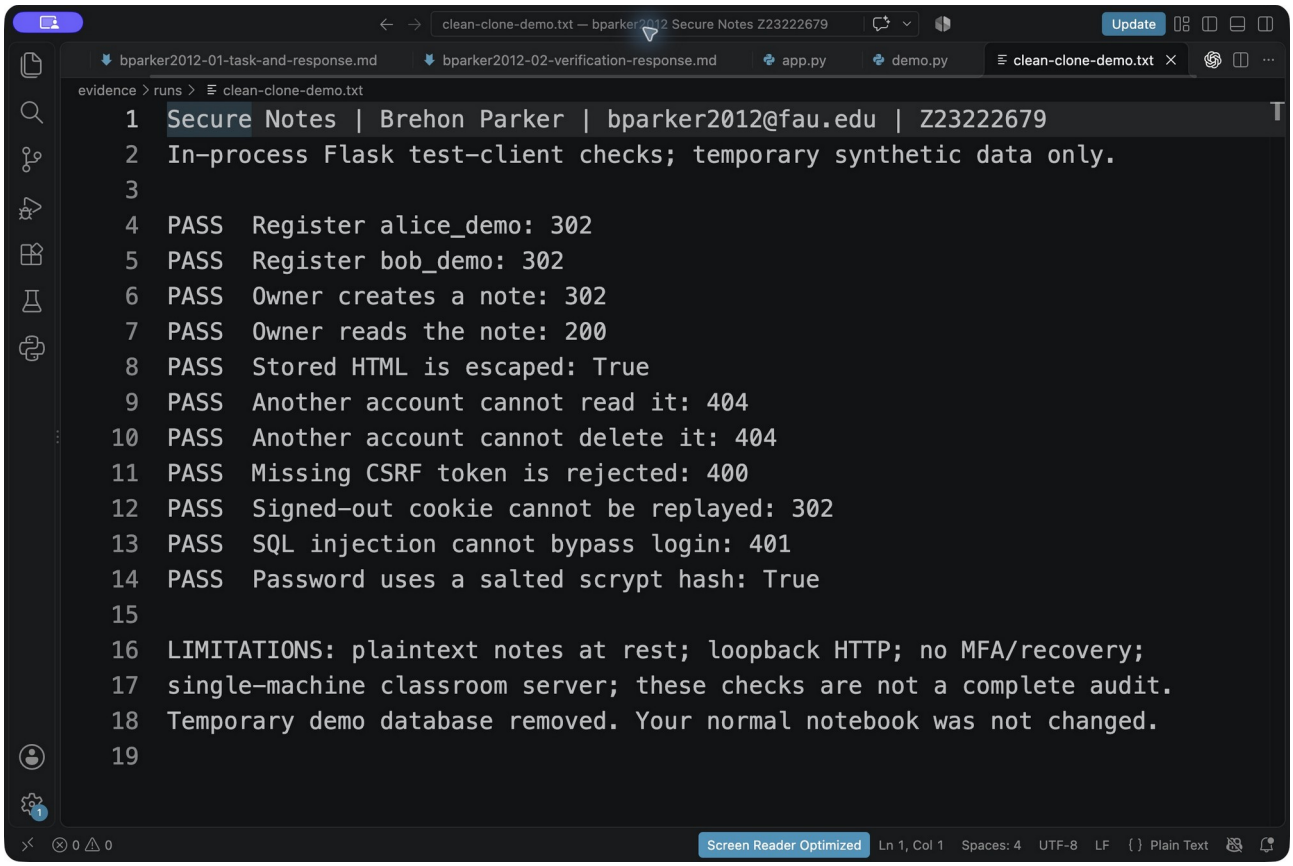


Figure 7. Captured output from the clean-clone in-process security demo, including expected denial responses.

The runtime dependency audit covered seven pinned packages and reported no known vulnerabilities at check time. Tests and dependency checks cover specific cases; they are not a complete security audit.

Browser demonstration

The application was also exercised in Chrome. A synthetic sample account registered successfully and created a private note. The notebook displays the saved content and presents editing and deletion controls. The complete create/read/edit/delete and sign-out flow was checked through real route handlers.

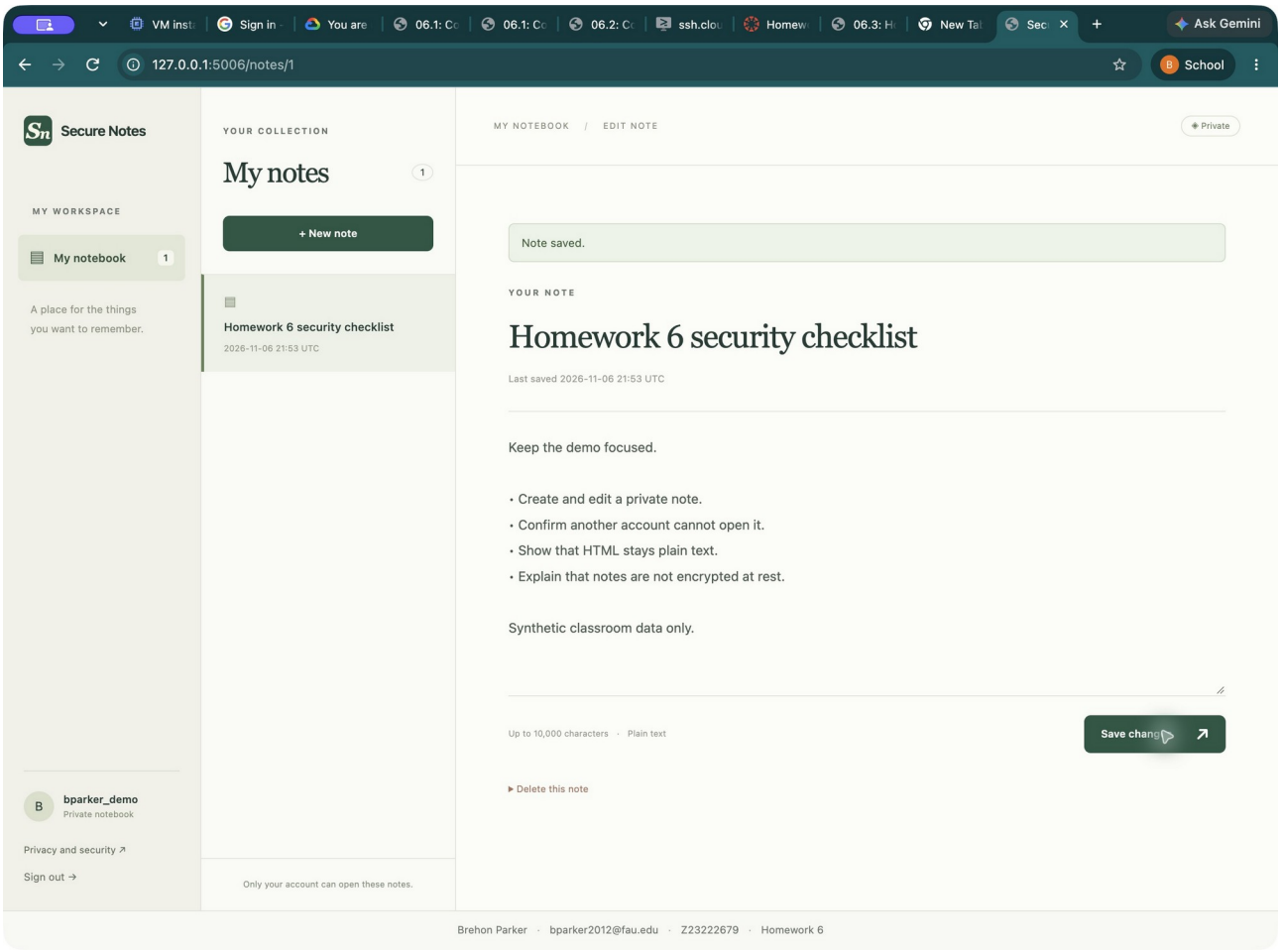


Figure 8. The running local notebook with a saved synthetic note and the student identity in the footer.

Capabilities and limitations

The app supports separate accounts and persistent private notes, with password hashing, parameterized SQL, output escaping, owner checks, CSRF verification, revocable sessions, request limits, and authentication rate limits. Note content is plaintext at rest. Filesystem access exposes it, and deleted data may remain in database pages or backups. The app has no MFA, password recovery, sharing, or attachments. Public deployment requires HTTPS, a production server, operational controls, and further security review.

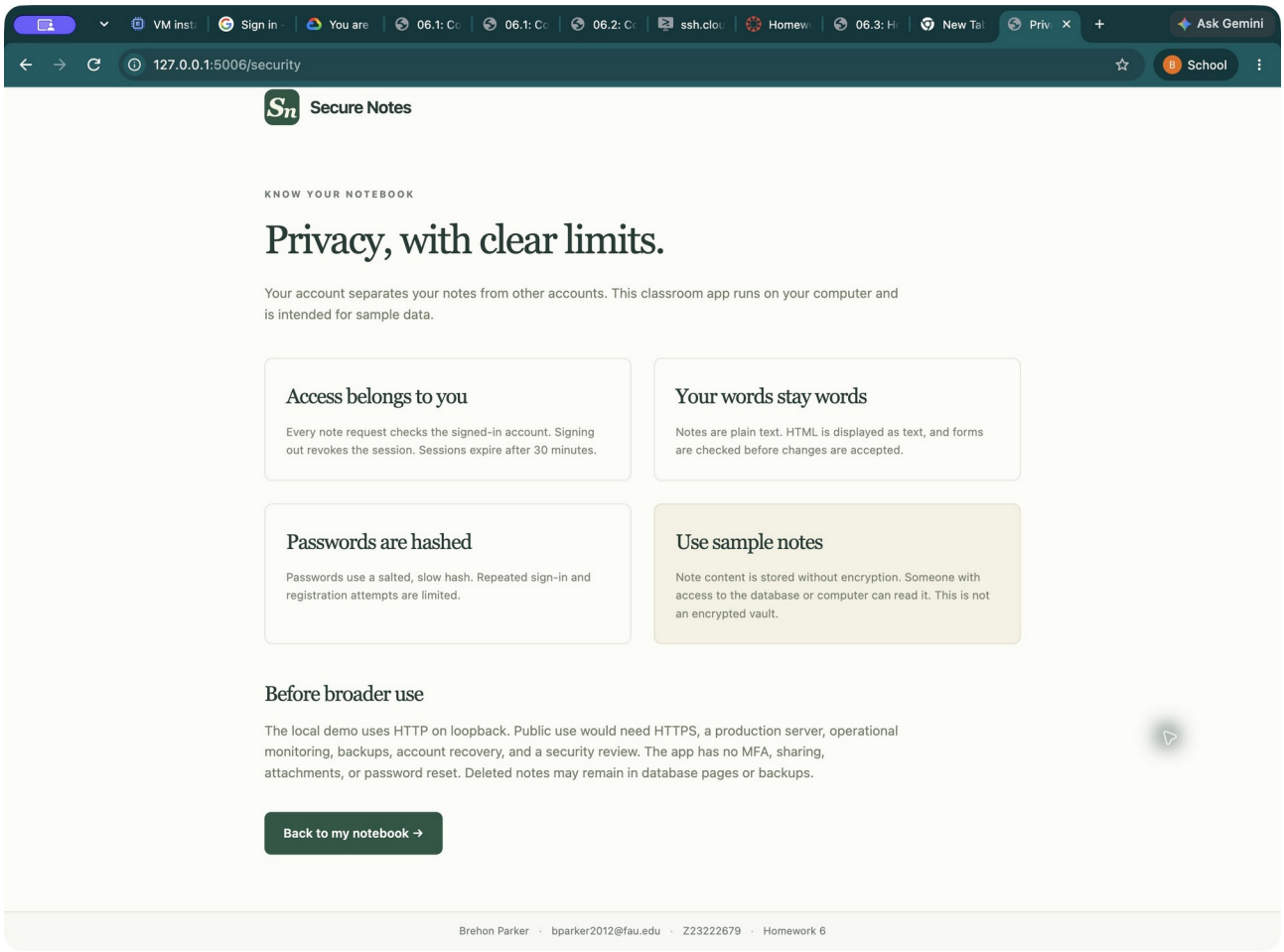


Figure 9. The application explains its privacy limits to the user, with the FAU ID visible.

Repository history and recording

The course repository records development in successive stages, rather than one final code dump: b9b0d7e scaffolds required files and generation prompts; 1e024a0 implements the app; e7b8303 adds security tests and fixes; 0f69a39 adds reproducible setup and recording guidance. Evidence and the report are committed separately after verification.

The recording follows this order: git clone, prompt overview, live prompt recreation in an empty workspace, security explanation, then functionality and limitations. The camera must be on initially and the narrated video must not exceed five minutes. SCREENCAST_SCRIPT.md supplies the exact commands and a target 4 minute 40 second narration.

Recording status: The student's live prompt replay, narrated recording, and unlisted YouTube upload remain to be completed.

The replay helper copies only prompts and dependency files into a new temporary folder. It never copies app.py, templates, CSS, tests, a database, or a signing key. Newly generated code can differ and must be tested and demonstrated as produced.

References

Assignment instructions: https://icardei.github.io/gensec-web/labs/G06.3_hw_wk6/index.html

Instructor homework template: https://icardei.github.io/gensec-web/homework-template.docx

Flask security guidance: https://flask.palletsprojects.com/en/stable/web-security/

Werkzeug password hashing: https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security